

DATA-260 Homework 5 – Report

Uday Patel

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16" (Ryzen 5 7640HS, 16GB RAM, RTX 4060 8GB, 512GB SSD).

Local model used: qwen3:8b via Ollama (Part 5 agent); sentence-transformers/all-MiniLM-L6-v2 (not used in this homework).

Repository: <https://github.com/UdayPate/data260-9871>

Tagged commit: 3c0acd2537128f596cf7ebbf12fc28235a72daa (tag: hw5)

<https://github.com/UdayPate/data260-9871/tree/3c0acd2537128f596cf7ebbf12fc28235a72daa>

Collaborator access: confirmed working for Sbnikitha and supriyaselvangesan.

Part 1.I – Database

Extended the existing `s9871_rel` database (already holding 5,001 `fixtures` rows from HW4) rather than creating a new service. The related entity is `teams`: one team is the home team of many fixtures. `fixtures` gained a unique code, a defaulted numeric field, and a foreign key to `teams` with ON DELETE RESTRICT.

```
class Team(Base):
    __tablename__ = "teams"
    id = Column(Integer, primary_key=True, autoincrement=True)
    team_name = Column(String(255), nullable=False)           # primary text field
    home_ground = Column(String(255), nullable=False)         # secondary text field
    team_code = Column(String(20), nullable=False, unique=True) # unique field, TM-####
    created_at = Column(DateTime, nullable=False, server_default=func.now())
    updated_at = Column(DateTime, nullable=False, server_default=func.now(),
                             onupdate=func.now())
    fixtures = relationship("Fixture", back_populates="home_team")

class Fixture(Base):
    __tablename__ = "fixtures"
    id = Column(Integer, primary_key=True, autoincrement=True)
    fixture_name = Column(String(255), nullable=False)
    teams_players = Column(String(255), nullable=False)
```

```

fixture_code = Column(String(24), nullable=False, unique=True) # FX-9871-####
spots_available = Column(Integer, nullable=False, server_default="22", default=22)
home_team_id = Column(Integer, ForeignKey("teams.id", ondelete="RESTRICT"),
                        nullable=False)
created_at = Column(DateTime, nullable=False, server_default=func.now())
updated_at = Column(DateTime, nullable=False, server_default=func.now(),
                    onupdate=func.now())
home_team = relationship("Team", back_populates="fixtures")

```

A fixture involves two sides, but the assignment asks for a single foreign key, so `home_team_id` deliberately models only the home side; the away side stays in the free-text `teams_players` column.

Because `fixtures` already held 5,001 rows, `migrate_hw05.py` adds the new columns NULL-able, backfills them deterministically from the primary key, then tightens to NOT NULL and adds the UNIQUE key and foreign key – `create_all()` cannot ALTER an existing table. Real schema after migration (no named tool requested a screenshot here, so the real console output substitutes):

```

CREATE TABLE 'teams' (
  'id' int NOT NULL AUTO_INCREMENT,
  'team_name' varchar(255) NOT NULL,
  'home_ground' varchar(255) NOT NULL,
  'team_code' varchar(20) NOT NULL,
  'created_at' datetime NOT NULL DEFAULT (now()),
  'updated_at' datetime NOT NULL DEFAULT (now()),
  PRIMARY KEY ('id'),
  UNIQUE KEY 'team_code' ('team_code')
) ENGINE=InnoDB AUTO_INCREMENT=13 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE 'fixtures' (
  'id' int NOT NULL AUTO_INCREMENT,
  'fixture_name' varchar(255) NOT NULL,
  'teams_players' varchar(255) NOT NULL,
  'fixture_code' varchar(24) NOT NULL,
  'spots_available' int NOT NULL DEFAULT '22',
  'home_team_id' int NOT NULL,
  'created_at' datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  'updated_at' datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY ('id'),
  UNIQUE KEY 'uq_fixtures_fixture_code' ('fixture_code'),
  KEY 'idx_fixture_name' ('fixture_name'),
  KEY 'fk_fixtures_home_team' ('home_team_id'),
  CONSTRAINT 'fk_fixtures_home_team' FOREIGN KEY ('home_team_id')
    REFERENCES 'teams' ('id') ON DELETE RESTRICT
) ENGINE=InnoDB AUTO_INCREMENT=5005 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

row count teams = 12
row count fixtures = 5001

```

Part 1.II – API Implementation

Full CRUD for both entities, pagination on the team listing, and a relationship-query endpoint (shown in the code below). Deleting a team that still has fixtures is refused at the API (409) and independently at the database level (the foreign key above):

```

@router.delete("/teams/{team_id}")
def delete_team(team_id: int, user: User = Depends(get_current_user),
               db: DBSession = Depends(get_db)):

```

```

team = db.query(Team).filter(Team.id == team_id).first()
if not team:
    raise HTTPException(status_code=404, detail=f"Team {team_id} not found")
dependents = db.query(Fixture).filter(Fixture.home_team_id == team_id).count()
if dependents:
    raise HTTPException(status_code=409, detail=(
        f"Team {team_id} still has {dependents} associated fixture(s) "
        "and cannot be deleted"))
db.delete(team)
db.commit()
return {"success": True, "deleted_id": team_id}

@router.get("/teams/{team_id}/fixtures")
def list_fixtures_for_team(team_id: int, user: User = Depends(get_current_user),
                           db: DBSession = Depends(get_db)):
    team = db.query(Team).filter(Team.id == team_id).first()
    if not team:
        raise HTTPException(status_code=404, detail=f"Team {team_id} not found")
    rows = db.query(Fixture).filter(Fixture.home_team_id == team_id)\
        .order_by(Fixture.id).all()
    return {"team": team_to_dict(team), "fixture_count": len(rows),
            "fixtures": [fixture_to_dict(f) for f in rows]}

```

Tested against the real server and the real `s9871_re1` database with a 15-request Postman collection, run twice consecutively (15/15 both times), fully self-cleaning.

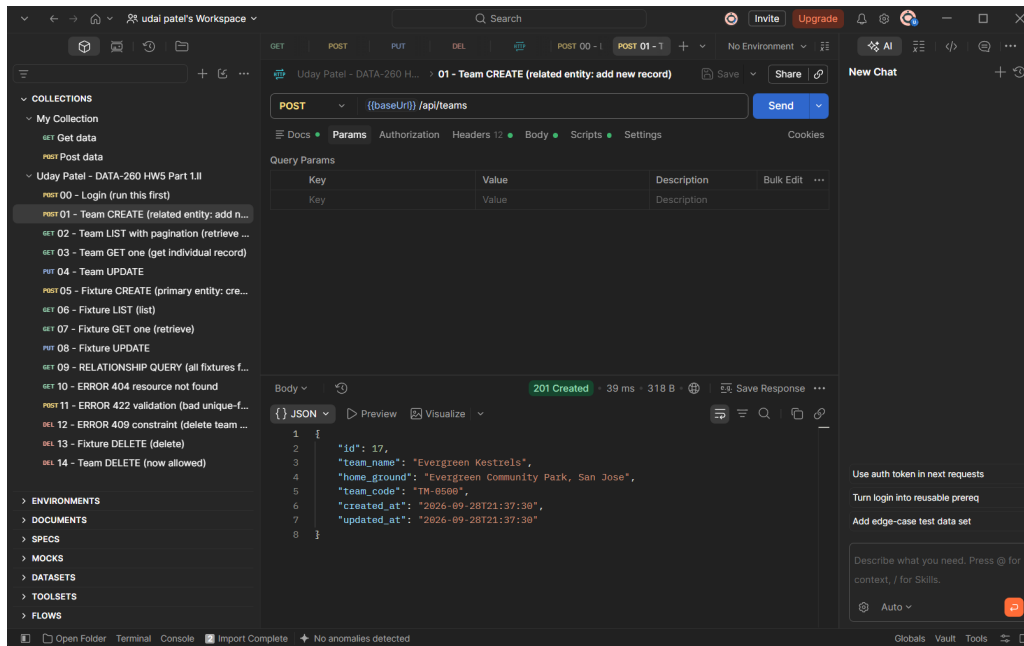


Figure 1: Postman – create team (201).

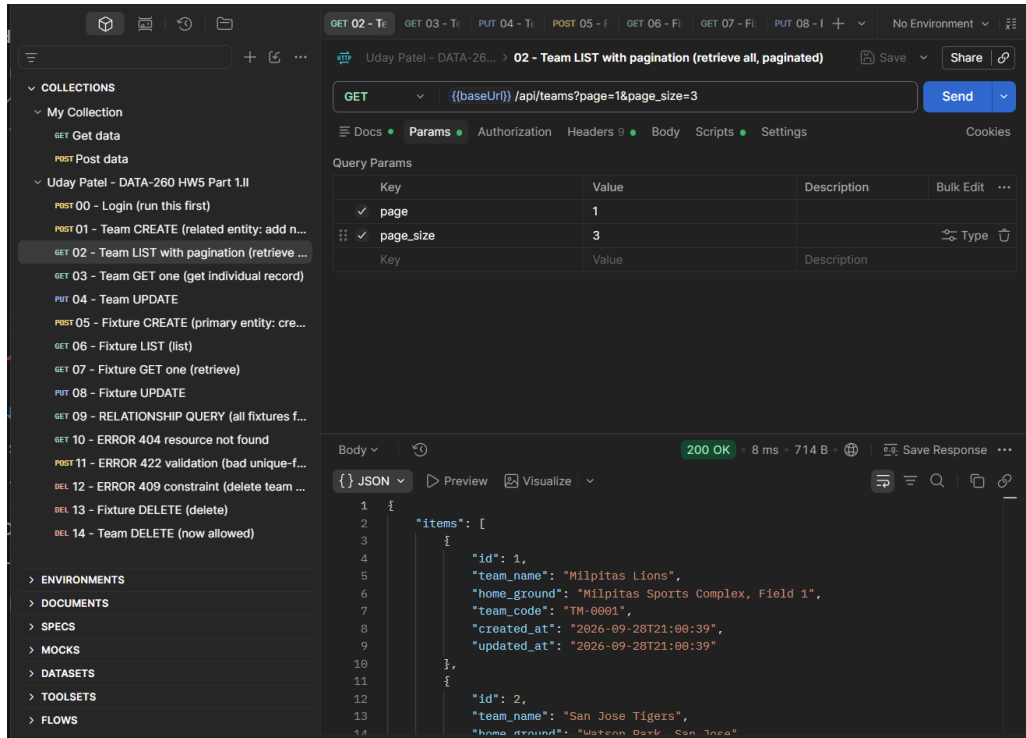


Figure 2: Postman – list teams, paginated (200).

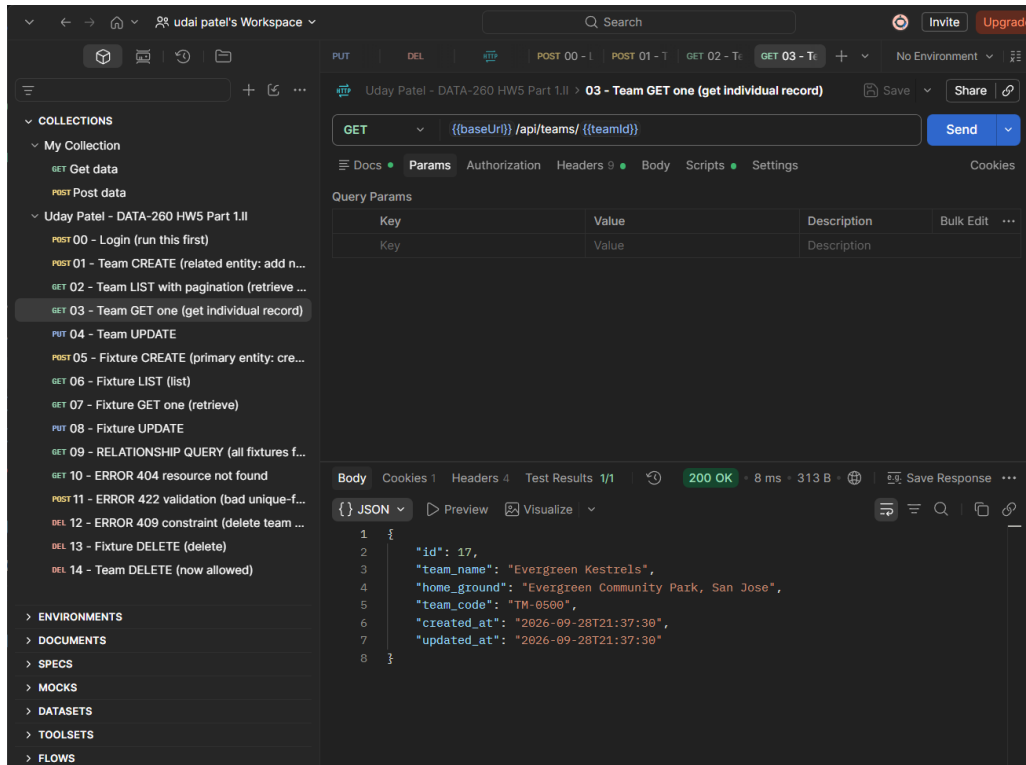


Figure 3: Postman – get one team (200).

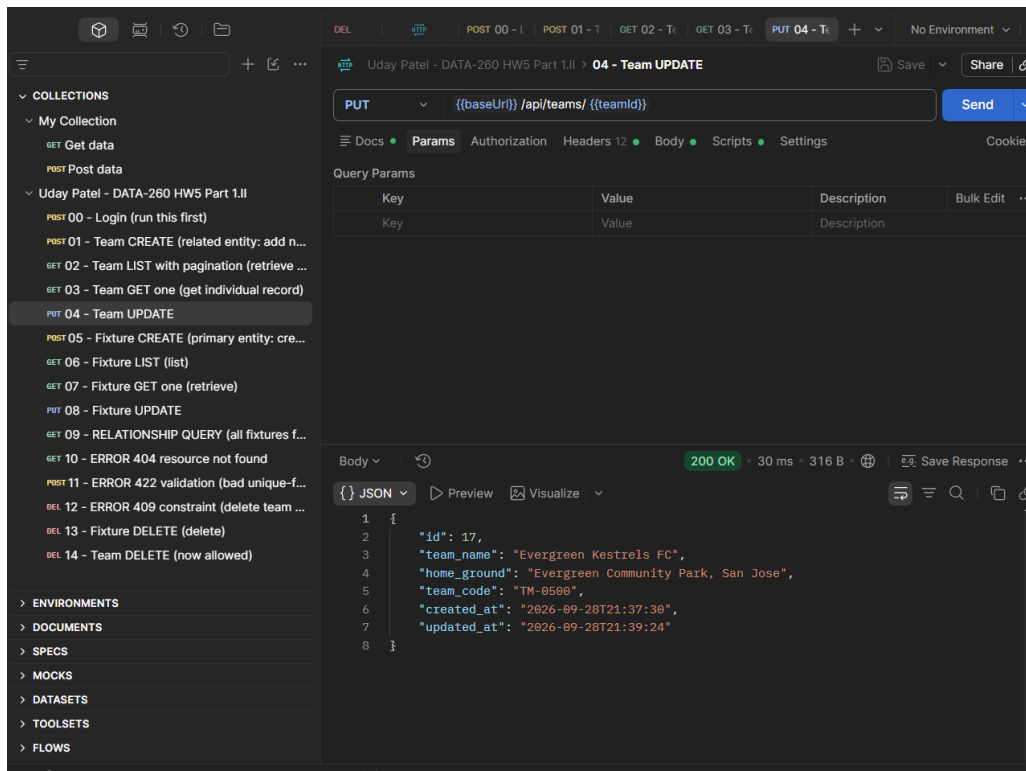


Figure 4: Postman – update team (200).

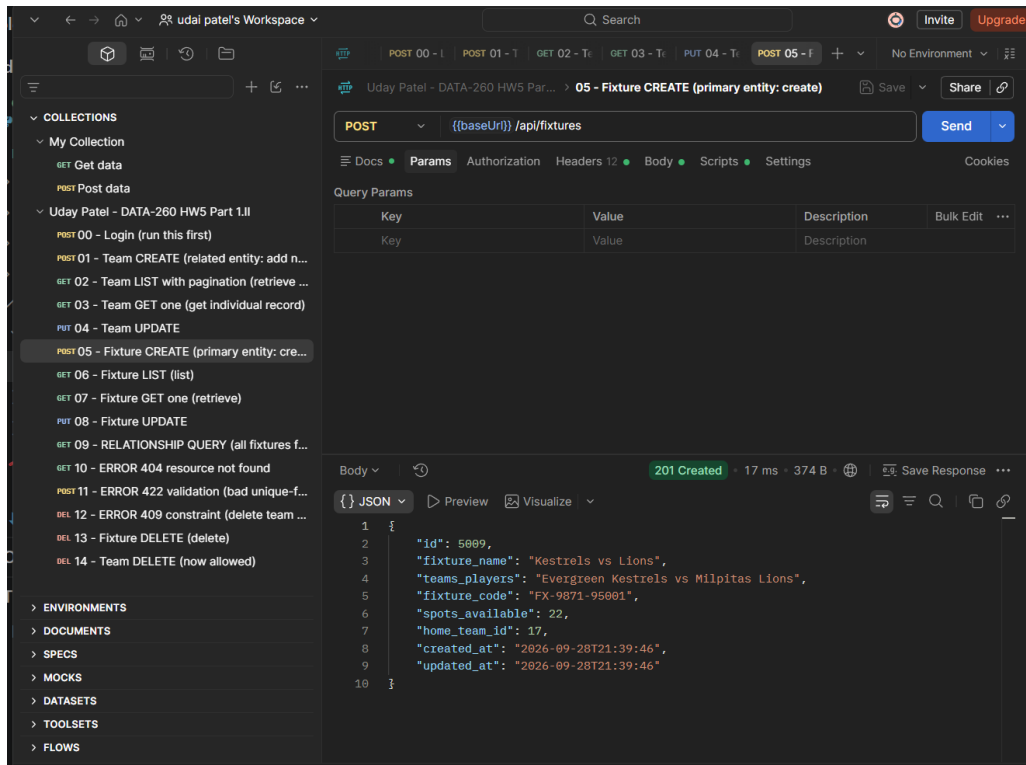


Figure 5: Postman – create fixture, spots_available defaulted to 22 (201).

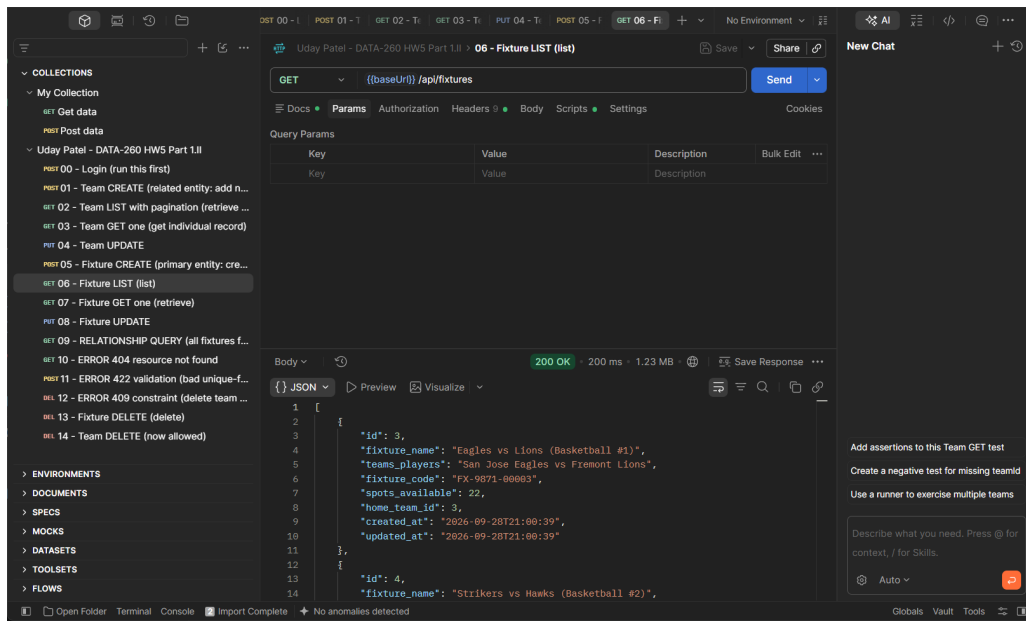


Figure 6: Postman – list fixtures (200).

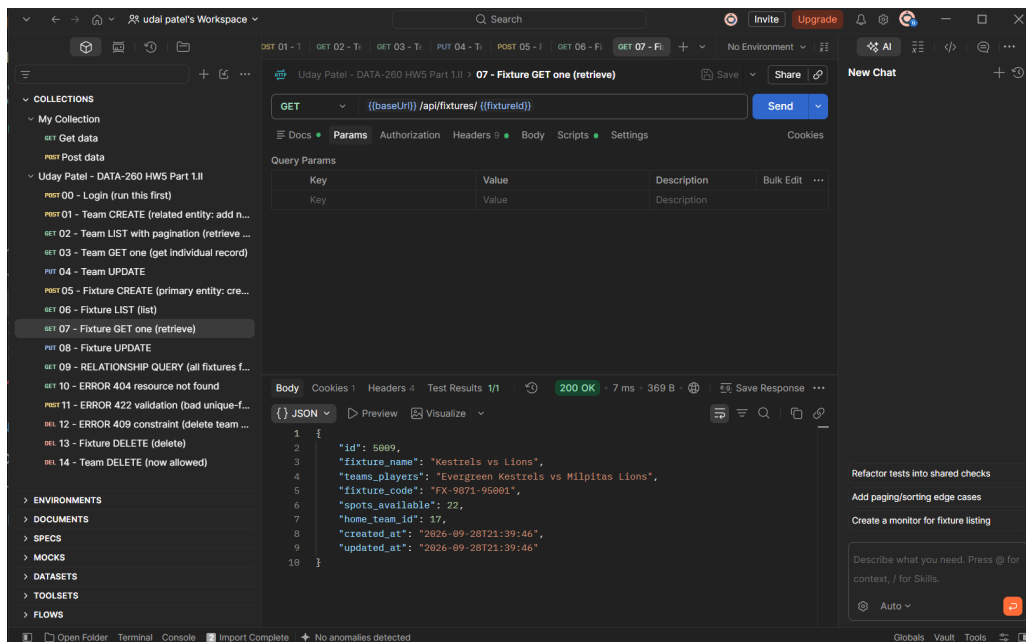


Figure 7: Postman – get one fixture (200).

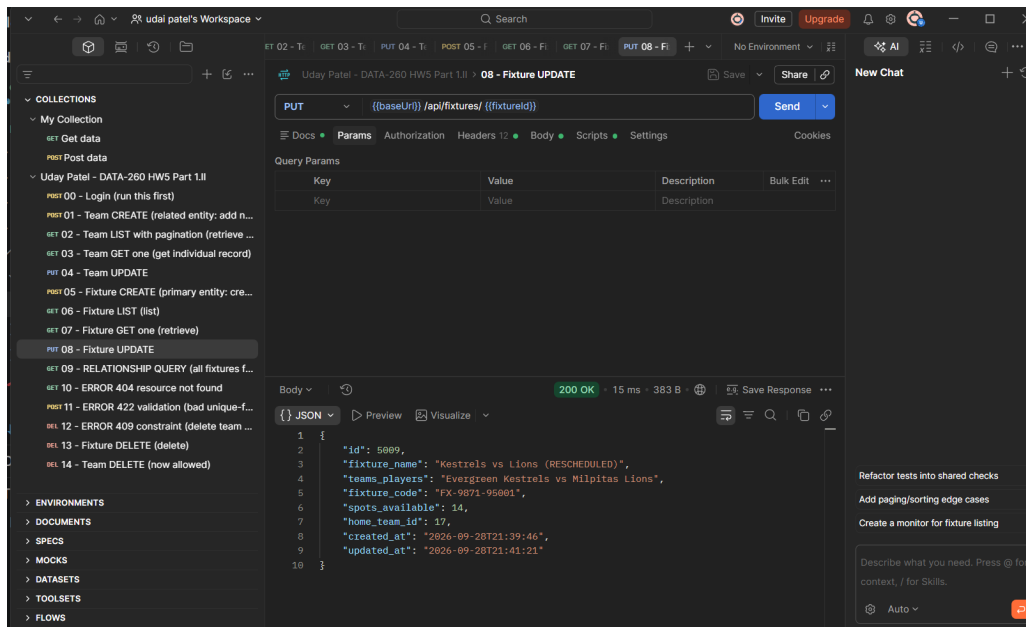


Figure 8: Postman – update fixture, spots 22 to 14 (200).

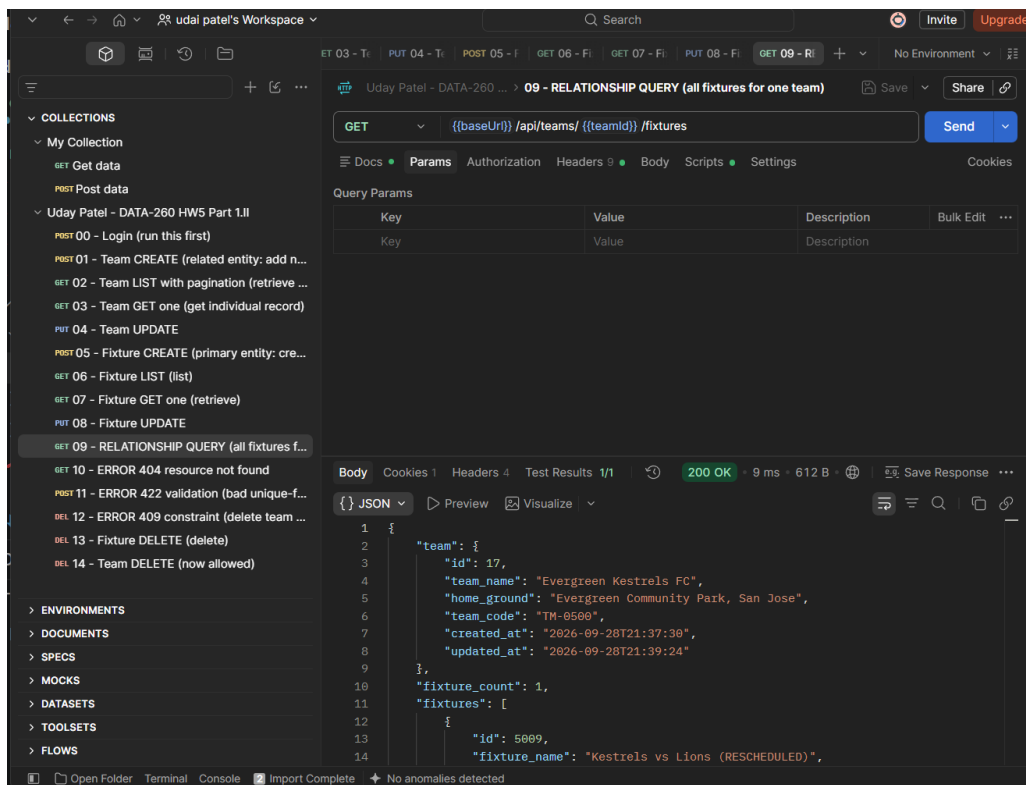


Figure 9: Postman – relationship query, fixtures for one team (200).

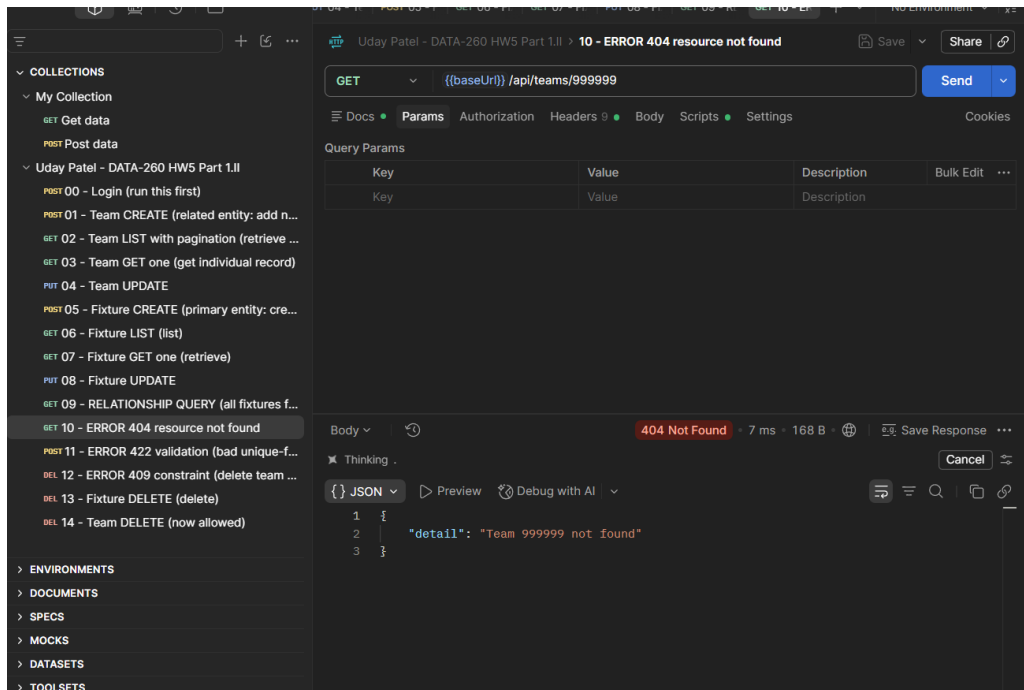


Figure 10: Postman – resource-not-found error (404).

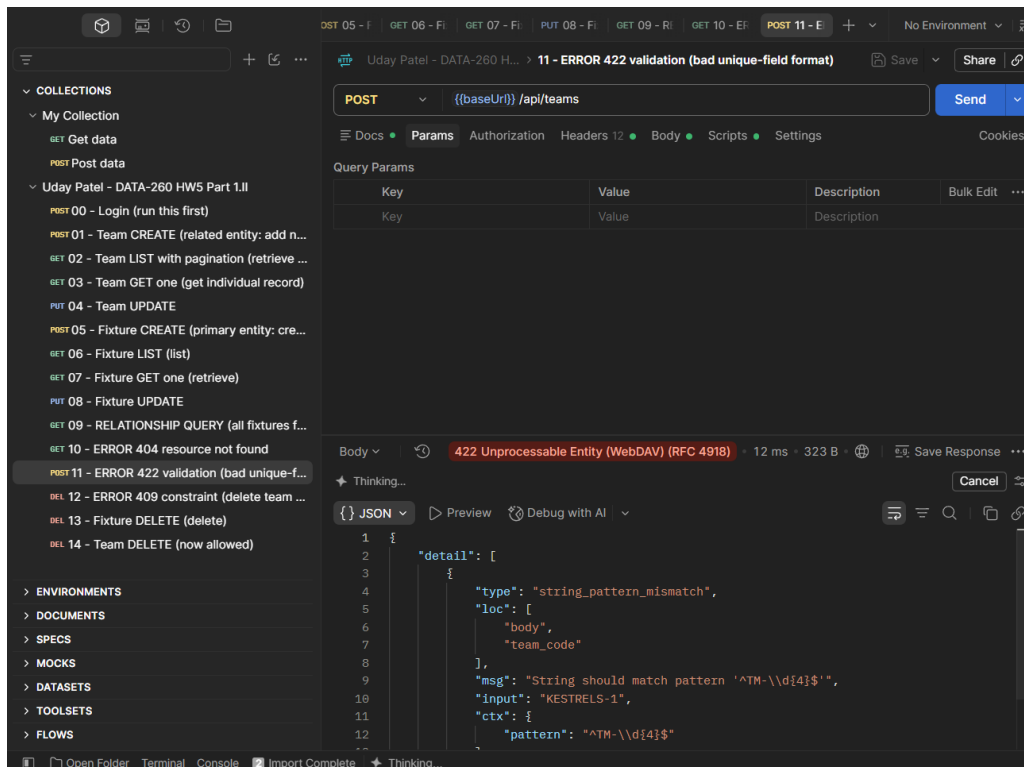


Figure 11: Postman – validation error on the unique-field format (422).

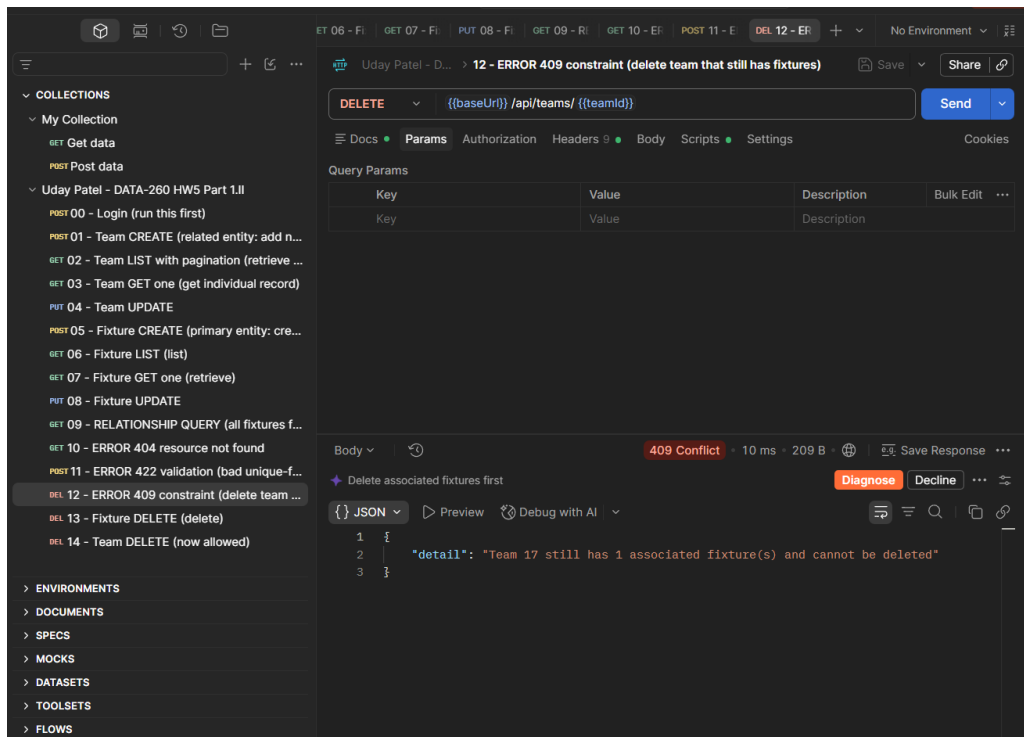


Figure 12: Postman – constraint violation, delete refused while fixtures exist (409).

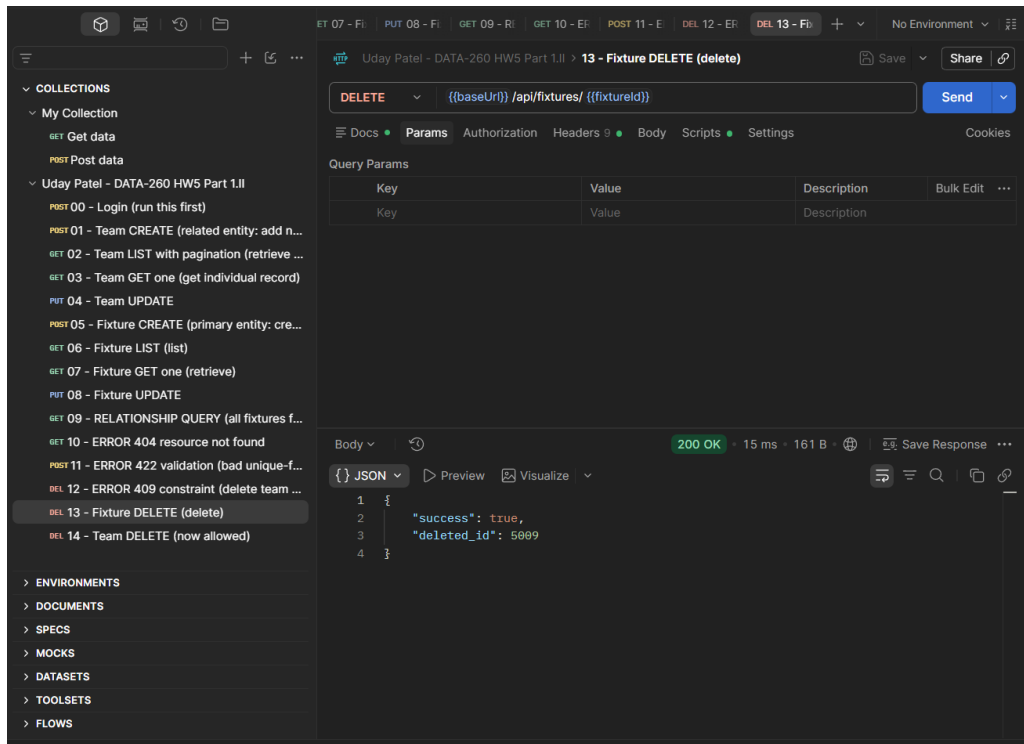


Figure 13: Postman – delete fixture (200).

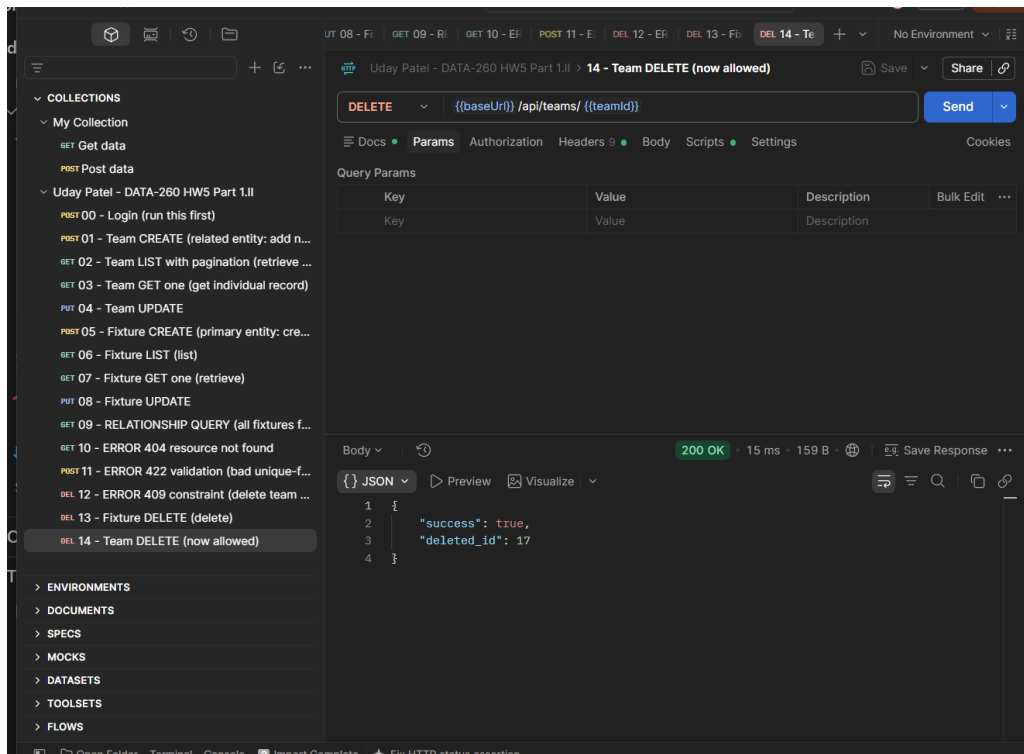


Figure 14: Postman – delete team, now allowed (200).

Part 1.III – Redux Client

Redux Toolkit store with a `fixtures` slice (the primary domain entity) and a small read-only `teams` slice for the home-team dropdown. Four async thunks call the FastAPI endpoints via axios.

```
export const store = configureStore({
  reducer: { fixtures: fixturesReducer, teams: teamsReducer },
});

export const createFixture = createAsyncThunk("fixtures/create",
  async (fixture, { rejectWithValue }) => {
    try {
      const response = await http.post("/fixtures", fixture);
      return response.data;
    } catch (err) { return rejectWithValue(errorMessage(err)); }
  });

export const updateFixture = createAsyncThunk("fixtures/update",
  async ({ id, changes }, { rejectWithValue }) => {
    try {
      const response = await http.put(`/fixtures/${id}`, changes);
      return response.data;
    } catch (err) { return rejectWithValue(errorMessage(err)); }
  });

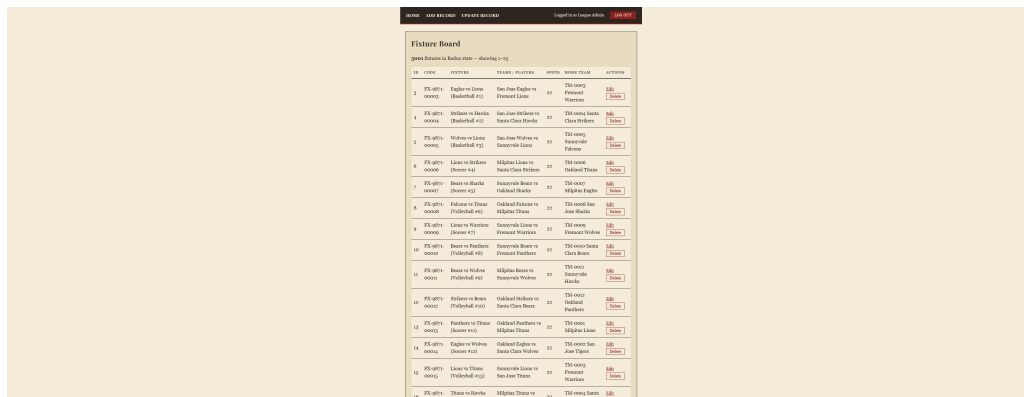
export const deleteFixture = createAsyncThunk("fixtures/delete",
  async (id, { rejectWithValue }) => {
    try {
      await http.delete(`/fixtures/${id}`);
      return Number(id); // reducer needs the id to drop the row
    } catch (err) { return rejectWithValue(errorMessage(err)); }
  });
```

Home reads everything from the store via `useSelector`, so the table updates itself after every create/update/delete with no manual refetch:

```
export default function Home({ user }) {
  const dispatch = useDispatch();
  const fixtures = useSelector(selectFixtures);
  const status = useSelector(selectFixturesStatus);
  const lastTouchedId = useSelector(selectLastTouchedId);
  const teams = useSelector(selectTeams);
  // ... pagination (25/page) and per-row Edit/Delete rendered below
```

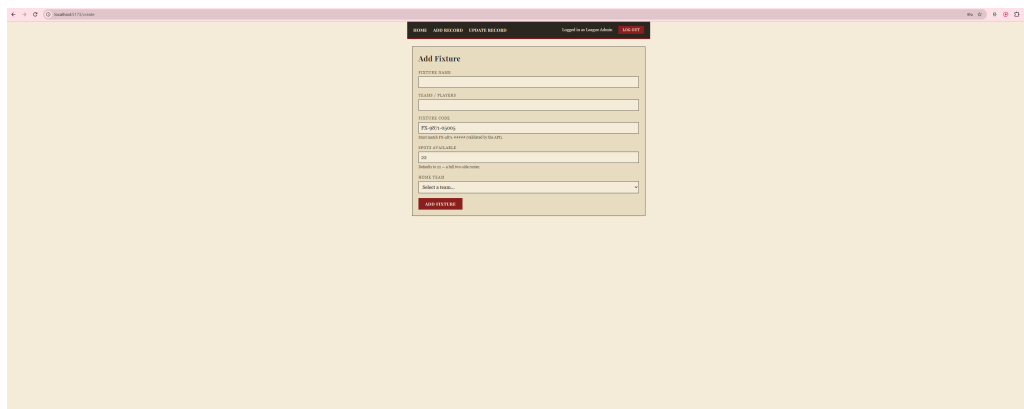
Verified with a real headless-browser test (Playwright): login, pagination, create + direct MySQL check, a forced 409 and 422 shown on the form, update by id with `updated_at` advancing, and per-row delete confirmed gone from both the table and MySQL – 22/22, twice consecutively.

Per CLAUDE.md’s established convention, “code snippet and UI output together, in the same frame” is satisfied here in the report layout: the relevant snippet directly above each screenshot, one caption below both.



The screenshot shows a web application interface with a navigation bar at the top containing links for HOME, ADD FIXTURE, and FIXTURE BOARD. The main content area is titled "Fixture board" and displays a table of fixtures. The table has columns for ID, CODE, FIXTURE NAME, STATUS, FIXTURE TYPE, and ACTIONS. It lists 20 fixtures, each with a unique ID and code, and a status of "ON". The fixtures are organized into groups, with some having a "FIXTURE TYPE" of "ON" and others "OFF". The table is paginated, showing 20 fixtures out of 20.

Figure 15: Home screen – reads the fixture list from Redux state (code above).



The screenshot shows a web application interface with a navigation bar at the top containing links for HOME, ADD FIXTURE, and FIXTURE BOARD. The main content area is titled "Add Fixture" and contains a form for adding a new fixture. The form has fields for FIXTURE NAME, FIXTURE TYPE, and a dropdown for SELECT A TEAM. There is also a "ADD FIXTURE" button at the bottom of the form.

Figure 16: Create screen – dispatches createFixture (code above).

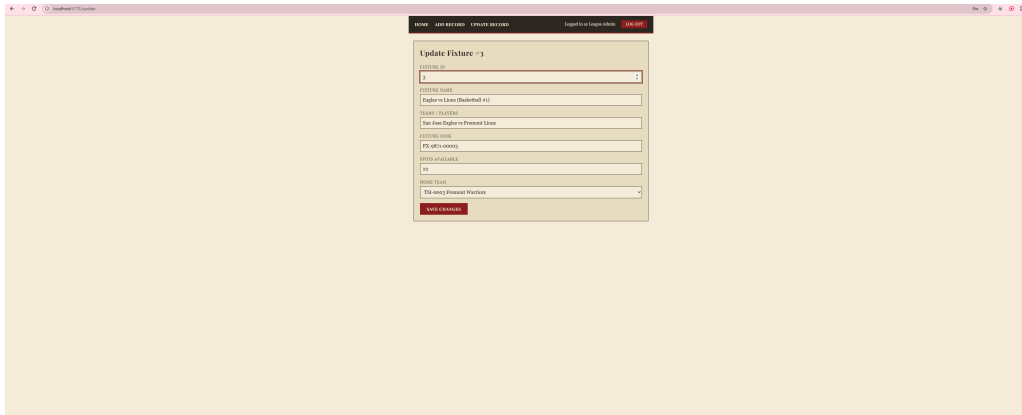


Figure 17: Update screen – select by id, dispatches `updateFixture` (code above).

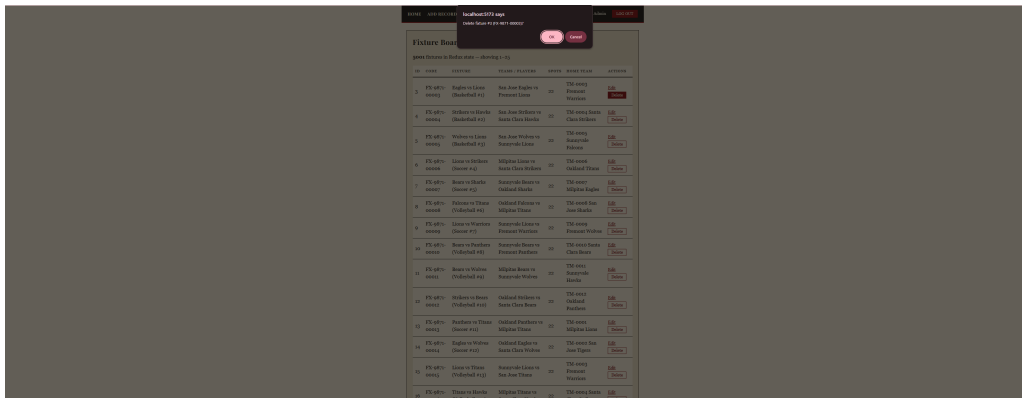


Figure 18: Delete – per-row delete button dispatches `deleteFixture` (code above).

Part 2.A – TheMealDB Tutorial MCP Server

A local MCP server named `meals` (code/`meals_server.py`), four tools over STDIO against the real TheMealDB API. mcp 2.2.0 renamed FastMCP to MCPServer; the API is otherwise the same.

```
def meal_details(id: str | int) -> dict | str:
    """Look up one meal by its TheMealDB id and return the full recipe."""
    meal_id = str(id).strip()
    if not meal_id.isdigit():
        raise ToolError(f"id must be numeric, got {id!r}")
    data = _get("lookup.php", {"i": meal_id})
    meals = data.get("meals")
    if not meals:
        return f"no meal found with id {meal_id}"
    return _full(meals[0])
```

Errors are raised as `ToolError` specifically – in mcp 2.2.0 that is the only exception type whose message reaches the MCP client; others are masked as a generic string, which would hide the real reason in the Inspector.

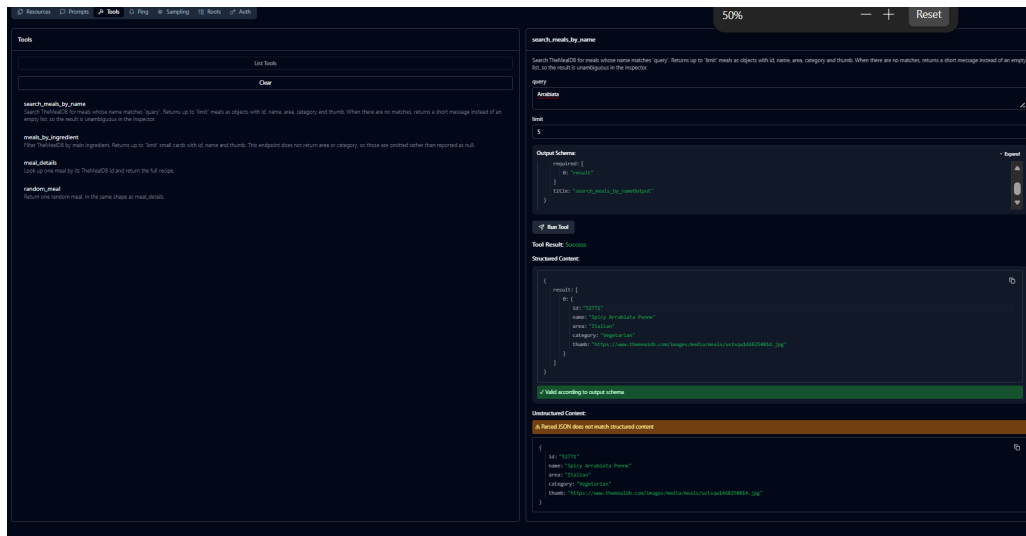


Figure 19: MCP Inspector – `search_meals_by_name`(Arrabiata, 5) – Spicy Arrabiata Penne.

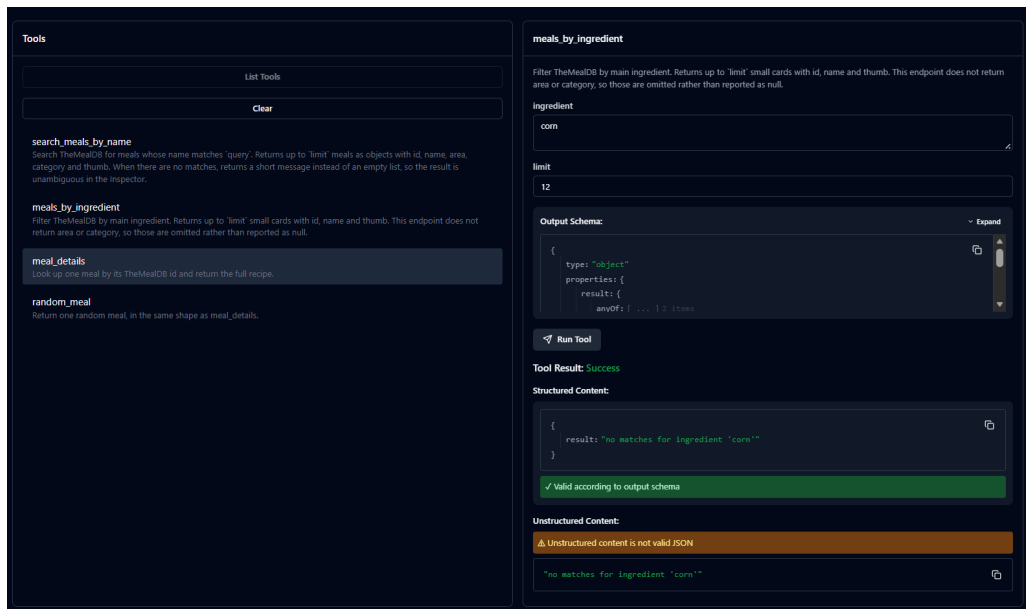


Figure 20: MCP Inspector – `meals_by_ingredient`.

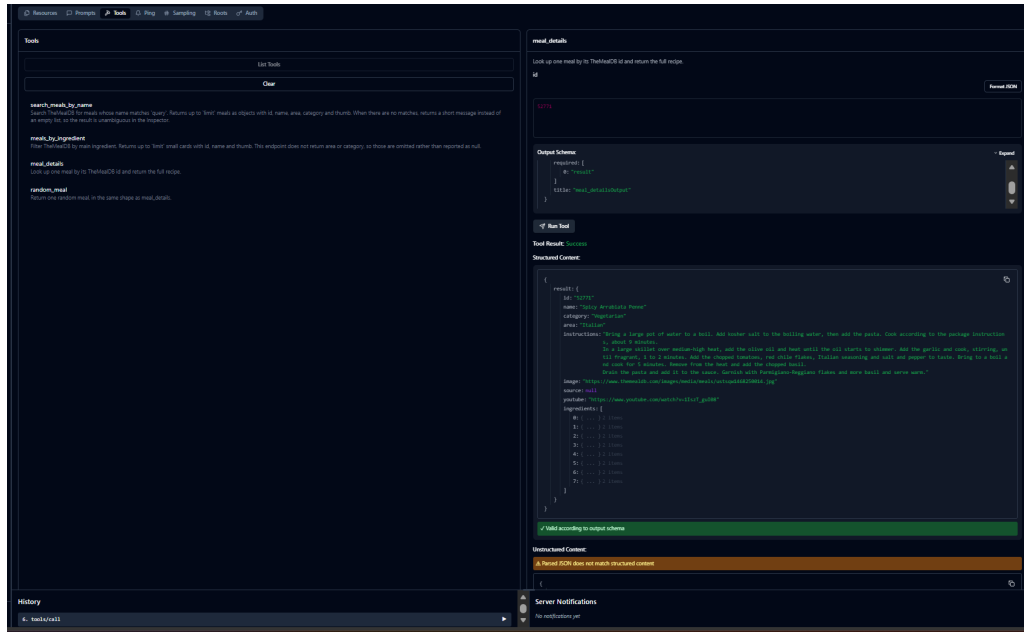


Figure 21: MCP Inspector – meal_details(52771), 8 ingredients.

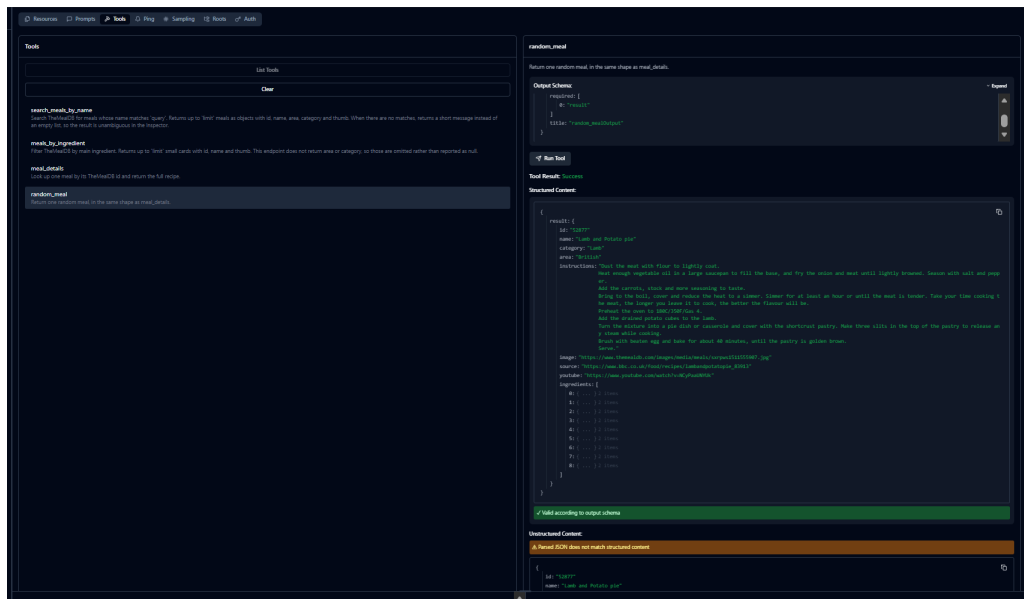


Figure 22: MCP Inspector – random_meal, same shape as meal_details.

Part 2.B – Domain MCP Server

A second server, domain, over the real s9871_rel data: exactly three tools, search / detail lookup / aggregate, all returning one shared envelope.

```
def result(data: Any = None, error: str | None = None) -> dict:
    """The one envelope shape every domain tool returns."""
    return {"ok": error is None, "data": data, "error": error}

@mcp.tool()
def team_fixture_counts(min_fixtures: int = 0) -> dict:
    """Aggregate: number of fixtures hosted by each team."""
    if min_fixtures < 0:
```

```

    return result(error=f"min_fixtures must be >= 0, got {min_fixtures}")
db = db_session_basede26()
try:
    teams = db.query(Team).order_by(Team.id).all()
    rows = db.query(Fixture.home_team_id, func.count(Fixture.id))\
        .group_by(Fixture.home_team_id).all()
    counts = dict(rows)
    data = [{"team_id": t.id, "team_code": t.team_code, "team_name": t.team_name,
            "fixture_count": counts.get(t.id, 0)}
            for t in teams if counts.get(t.id, 0) >= min_fixtures]
    return result(data=data)
finally:
    db.close()

```

Validated in MCP Inspector with one successful and one intentionally invalid call per tool, against the live database:

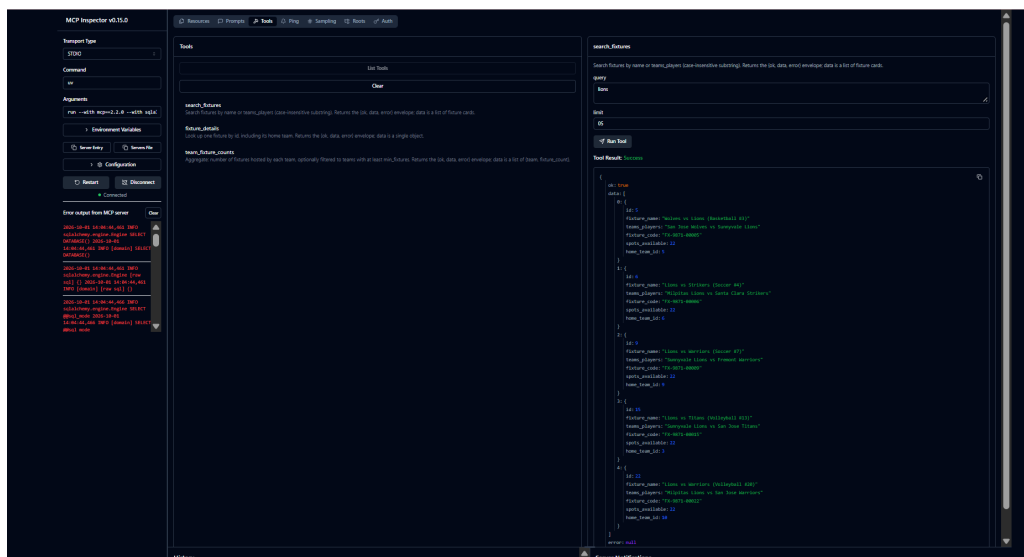


Figure 23: `search_fixtures("lions")` – ok: true, 5 real matches.

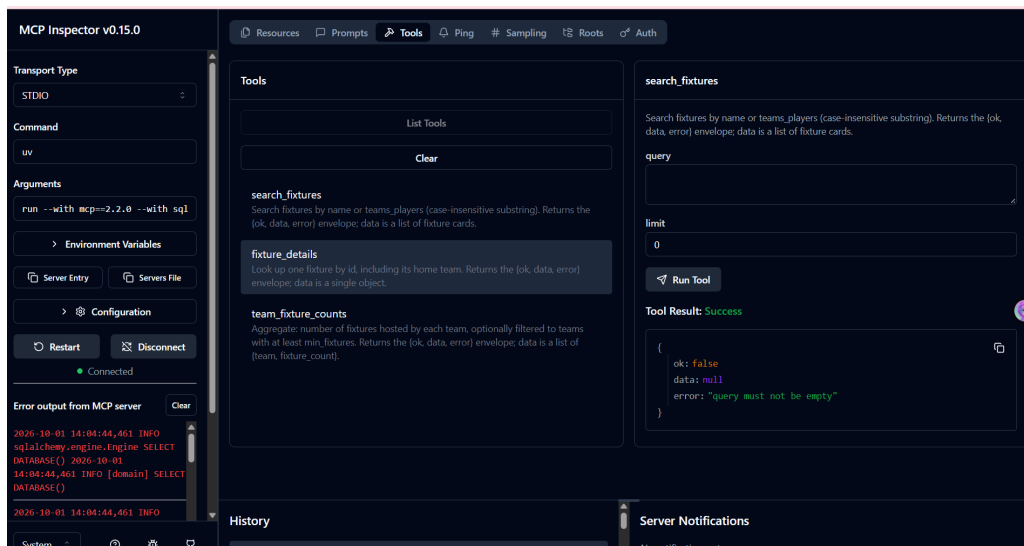


Figure 24: `search_fixtures("")` – ok: false, "query must not be empty".

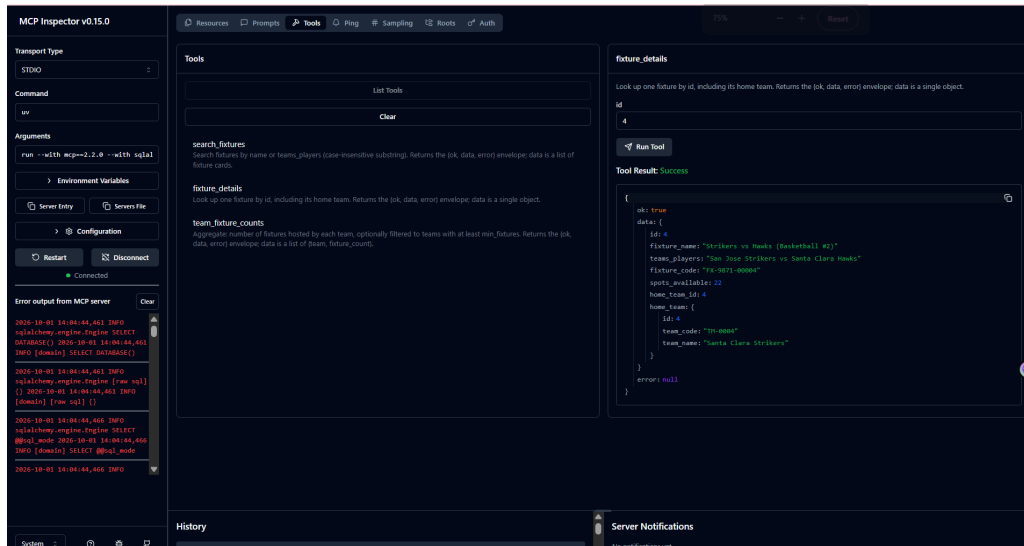


Figure 25: `fixture_details(4)` – `ok: true`, nested `home_team`.

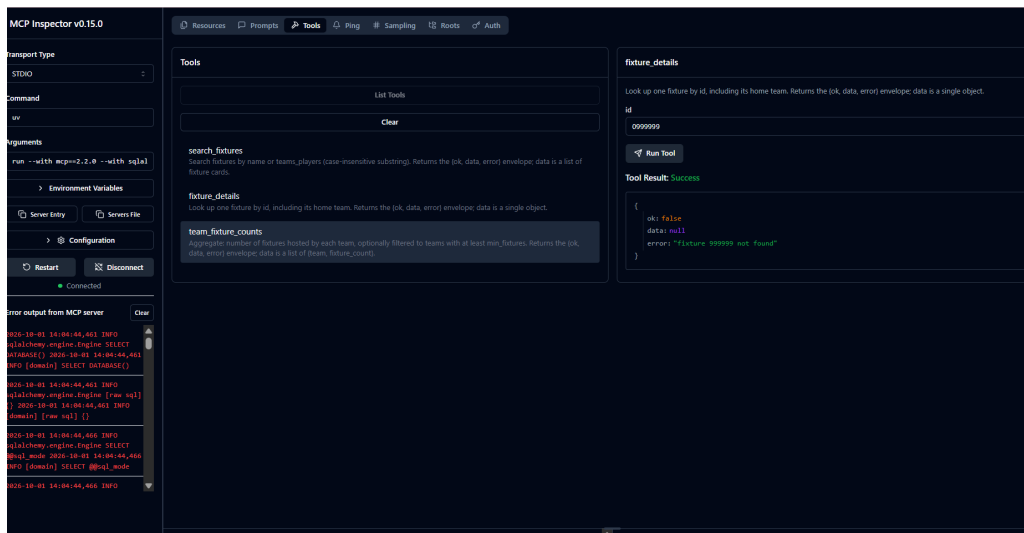


Figure 26: `fixture_details(999999)` – `ok: false`, "fixture 999999 not found".

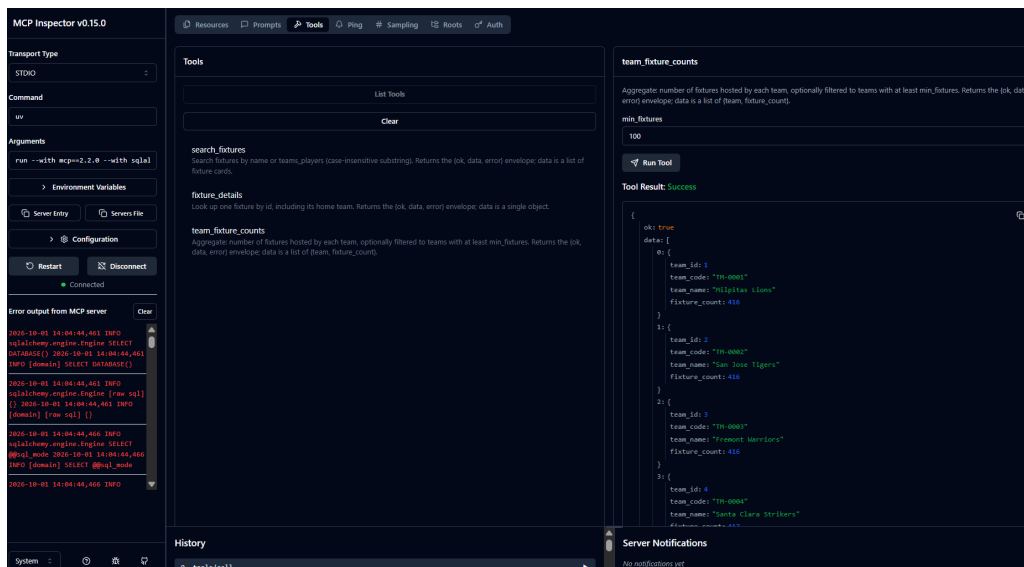


Figure 27: `team_fixture_counts(min_fixtures=100)` – `ok: true`, real per-team counts.

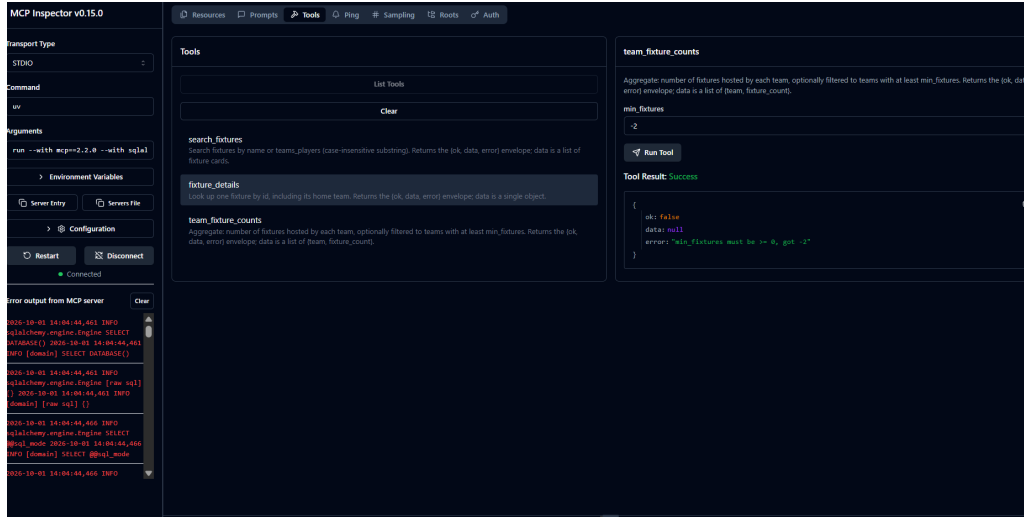


Figure 28: `team_fixture_counts(min_fixtures=-2)` – `ok: false`, "`min_fixtures must be >= 0, got -2`".

Part 3 – Tool Contracts Under Stress

3.18 – expected schema, rejected input, returned error (same three tools and rejected calls as Part 2.B above):

Tool	Expected schema	Rejected input	Returned error
<code>search_fixtures</code>	<code>{query: string (req.), limit: int=10}</code>	<code>{"query": ""}</code>	"query must not be empty"
<code>fixture_details</code>	<code>{id: integer (req.)}</code>	<code>{"id": 999999}</code>	"fixture 999999 not found"
<code>team_fixture_counts</code>	<code>{min_fixtures: int=0}</code>	<code>{"min_fixtures": -2}</code>	"min_fixtures must be >= 0, got -2"

3.19 – retry/timeout policy, each attempt in a worker thread so the timeout is a real enforced cutoff, not just a latency measurement:

```
def call_with_retry(operation, *, max_retries=3, base_delay_s=0.05,
                    max_delay_s=1.0, timeout_s=2.0) -> dict:
    start = time.monotonic()
    last_err = None
    for attempt in range(max_retries + 1):
        future = _executor.submit(operation)
        try:
            data = future.result(timeout=timeout_s)
            return {"ok": True, "data": data, "error": None, "attempts": attempt + 1,
                    "latency_ms": (time.monotonic() - start) * 1000}
        except FutureTimeoutError:
            last_err = f"timed out after {timeout_s}s"
        except Exception as exc:
            last_err = str(exc)
        if attempt < max_retries:
            time.sleep(min(base_delay_s * (2**attempt), max_delay_s))
    return {"ok": False, "data": None, "error": last_err, "attempts": max_retries + 1,
            "latency_ms": (time.monotonic() - start) * 1000}
```

Demonstrated, real output:

1. success on first attempt: {"ok": true, "data": {"status": "ok"}, "error": null, "attempts": 1, "latency_ms": 0.0}
2. failure then success on retry: {"ok": true, "data": {"status": "ok", "succeeded_on_attempt": 2}, "error": null, "attempts": 2, "latency_ms": 47.0}
3. failure after all retries: {"ok": false, "data": null, "error": "operation always fails", "attempts": 4, "latency_ms": 156.0}

3.20 – VERIFY_SEED fault injection, 150 calls (50 per rate), VERIFY_SEED = 269871, raw data in reports/hw05/raw/:

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100%	13.1	31.0
20%	100%	28.4	187.0
50%	94%	80.0	428.4

With max_retries=3 (4 attempts total), the policy fully absorbs 0% and 20% injected failure – failing 4 attempts in a row at 20% has probability $0.2^4 \approx 0.16\%$. At 50%, failing all 4 has probability $0.5^4 = 6.25\%$, matching the observed 6% failure almost exactly. p99 grows much faster than the mean because it is dominated by the calls that need several retries with exponential backoff.

Interactive vs. batch: suitable for an interactive assistant up to about 20% failure – every call still succeeds and p99 stays under 200ms. At 50%, a p99 of 428ms feels sluggish and 1 in 17 requests still fails outright. For batch processing, increase max_retries (5–6) and raise max_delay_s (5–10s) so a transient outage gets more chances to clear – worse worst-case latency traded for a higher overall success rate, the right trade when nothing is blocking an interactive user.

Part 4 – One Safe Tool Entry Point and Offline Tests

execute_tool(name, inputs) is the single function Part 5’s agent calls. It dispatches through the retry policy above and always returns a JSON string in the {ok, data, error} shape – never raises, even on an unknown tool name or a missing argument.

```
def execute_tool(name: str, inputs: dict, tools: dict | None = None) -> str:
    registry = tools or TOOLS
    fn = registry.get(name)
    if fn is None:
        return json.dumps({"ok": False, "data": None,
                          "error": f"unknown tool: {name!r} (known: {sorted(registry)})"})

    if name == "search_fixtures":
        # Part 5.I safety rule
        query = (inputs.get("query") or "").strip()
        if 0 < len(query) < 3:
            return json.dumps({"ok": False, "data": None,
                              "error": "query too broad: must be at least 3 characters"})

    def call():
        return fn(**inputs)

    outcome = call_with_retry(call, max_retries=2, base_delay_s=0.05, timeout_s=5.0)
    envelope = outcome["data"] if outcome["ok"] else \
        {"ok": False, "data": None, "error": outcome["error"]}
    return json.dumps(envelope)
```

A Python test runner using assert-backed checks, 7 tests covering one valid and one invalid input per domain tool plus execute_tool’s own unknown-tool-name dispatch, via dependency injection (fake tool callables, no MySQL, no LLM, fully offline and repeatable):

```

Windows PowerShell
PS C:\Users\patel\data260-9871\code> .\venv\Scripts\python.exe test_execute_tool.py
PASS search_fixtures: valid input -> ok=true
PASS search_fixtures: invalid (empty query) -> ok=false, documented error
PASS fixture_details: valid input -> ok=true
PASS fixture_details: invalid (unknown id) -> ok=false, documented error
PASS team_fixture_counts: valid input -> ok=true
PASS team_fixture_counts: invalid (negative min_fixtures) -> ok=false, documented error
PASS execute_tool: unknown tool name -> ok=false, no crash

7/7 tests passed
PS C:\Users\patel\data260-9871\code>

```

Figure 29: terminal – Part 4 offline test runner, 7/7, PASS/FAIL per test and final summary.

Part 5 – Agent Loop, Safety Rule, and Reflection

I. Safety rule – a `search_fixtures` query under 3 characters is rejected inside `execute_tool`, before the domain tool or any DB call runs:

```

allowed: query="Lions" -> {"ok": true, "data": [...2 matches...], "error": null}
blocked: query="a"      -> {"ok": false, "data": null,
                           "error": "query too broad: must be at least 3 characters"}

```

II. Agent loop – `run_agent(user_input)` asks qwen3:8b (through the existing `ModelClient` adapter from HW1/2) each turn to either call one of the three tools or give a final answer, in a strict JSON protocol; a turn counter enforces `max_steps`; every step and the final stop reason are logged to `agent_runs.jsonl`.

```

class MockModel:
    """Always asks for another tool call - guarantees max_steps is hit."""
    def complete(self, messages, tools=None):
        return json.dumps({"action": "tool", "tool": "team_fixture_counts", "inputs": {}})

while step < max_steps:
    step += 1
    raw = model.complete(messages)
    parsed = _extract_json(raw) # tolerant of <think> blocks / stray prose
    if parsed.get("action") == "final":
        final_answer = parsed.get("answer", ""); stop_reason = "final_answer"; break
    if parsed.get("action") == "tool":
        result = json.loads(execute_tool(parsed["tool"], parsed.get("inputs") or {},
                                         tools=tools))

        tool_call_count += 1
        log({"step": step, "tool": parsed["tool"], "result": result})
        if not result.get("ok") and "query too broad" in (result.get("error") or ""):
            stop_reason = "safety_rule_block"; break
        messages.append({"role": "user",
                        "content": f"Tool result: {json.dumps(result)}"})
        continue
# loop exits with stop_reason = "max_steps_ceiling" if nothing else set it

```

III. Additional offline tests – two tests added to the existing Part 4 suite (9 total), both fully offline (fake tools, `MockModel`, no API key/live model/database):

```

PASS search_fixtures: valid input -> ok=true
PASS search_fixtures: invalid (empty query) -> ok=false, documented error
PASS fixture_details: valid input -> ok=true
PASS fixture_details: invalid (unknown id) -> ok=false, documented error
PASS team_fixture_counts: valid input -> ok=true
PASS team_fixture_counts: invalid (negative min_fixtures) -> ok=false, documented error
PASS execute_tool: unknown tool name -> ok=false, no crash
PASS execute_tool: safety rule blocks a too-broad query, no crash
PASS run_agent + MockModel: stops at max_steps, offline

9/9 tests passed

```

IV. Metrics – 4 real scenarios against local qwen3:8b:

Scenario	Stop reason	Steps	Tool calls
A – detail lookup	final_answer	2	1
B – aggregate + follow-up reasoning	final_answer	2	1
C – asked to run a too-short search query	final_answer	1	0
D – forced ceiling (max_steps=1)	max_steps_ceiling	1	1

Honestly reported: Scenario A answered correctly (fixture 4’s real code and spots). Scenario B’s tool call succeeded with real data, but the model’s final answer was nonsensical – it claimed the conversation was “stuck in a loop” rather than reporting the aggregate result; this is a genuine model failure, not a bug in `execute_tool/run_agent`, and is reported as-is rather than re-run for a cleaner result. Scenario C is notable: the model read the system prompt’s stated 3-character rule and declined the tool call itself (`tool_call_count=0`), so the live agent never actually exercised the safety-rule code path – that path is demonstrated directly above instead. Scenario D is a clean, deliberately forced `max_steps_ceiling`.

V. Reflection (REFLECTION.md, run_id 45743a9ea8d1, scenario D):

On turn 1, `run_agent` sent the system prompt and the question “Look up fixture id 10 and summarize it for me.” to qwen3:8b. The model replied `{"action": "tool", "tool": "fixture_details", "inputs": {...}}`, correctly recognizing the question needed a tool call, not a guess. `execute_tool` ran `fixture_details(id=10)` through the retry policy, succeeding on the first attempt and returning the real row: “Bears vs Panthers (Volleyball #8)”, code FX-9871-00010, 22 spots available, home team Santa Clara Bears. That envelope was logged as the step record and appended to the conversation for the model’s next turn.

That is where the run stopped. `max_steps` was deliberately set to 1, so the loop condition was already false after the one turn that produced the tool call – no second turn remained for the model to read the result and write a summary. `run_agent` exits, finds no final answer and no safety-rule block, and falls back to `stop_reason = "max_steps_ceiling"`. The final log record shows exactly this: `tool_call_count: 1, steps: 1, final_answer: null` – the agent had the right data but ran out of turns before reporting it.

This illustrates why `max_steps` matters even when nothing goes wrong: a correctly-behaving agent can still hit a step budget too low for the task. This question needed at least two turns – call the tool, then read the result back – and `max_steps=1` wasn’t enough by design. The harness stopped cleanly rather than looping forever or guessing without the data it had just fetched.